

Cours complet Laravel : Migrations, Connexions, Routes et Modèles

Partie I — Les Migrations

1. Qu'est-ce qu'une migration ?
2. Anatomie d'un fichier de migration
3. Les types de colonnes les plus utilisés
4. Les modificateurs de colonnes
5. Les clés étrangères (relations entre tables)
6. Modifier une table existante
7. Les commandes Artisan indispensables
8. Créer une migration "avec le modèle" (bonne pratique)
9. La table `migrations` : comment Laravel sait ce qui a déjà été exécuté
10. Exemple concret : construire un mini-schéma pour un projet type "stagilog"
11. Bonnes pratiques à retenir
12. Aller plus loin sur les migrations

Partie II — La connexion à la base de données

13. Le fichier `.env`
14. Le fichier `config/database.php`
15. Tester et utiliser la connexion

Partie III — Le routage (Routes)

16. Où définir les routes ?
17. Routes de base
18. Paramètres de route
19. Routes nommées
20. Routes ressources (Resource Routes)
21. Groupes de routes
22. Middlewares sur les routes
23. Voir toutes les routes de l'application

Partie IV — Les Modèles Eloquent

24. Qu'est-ce qu'Eloquent ?
25. Créer un modèle
26. Structure de base d'un modèle
27. Opérations CRUD avec Eloquent
28. Le Query Builder d'Eloquent (requêtes avancées)
29. Les relations entre modèles
30. Accesseurs et mutateurs (personnaliser la lecture/écriture)
31. Scopes locaux (requêtes réutilisables)

Partie V — Comment tout s'articule (le flux MVC)

32. De la requête HTTP à la réponse

Conclusion et ressources

Cours complet Laravel : Migrations, Connexions, Routes et Modèles

Ce document couvre quatre piliers fondamentaux d'un projet Laravel : la gestion de la base de données via les **migrations**, la **connexion** à la base de données, le **routing** des requêtes HTTP, et les **modèles Eloquent**. À la fin, tu auras une vision complète du flux : une requête arrive → une route l'attrape → un contrôleur/modèle interroge la base → une réponse est renvoyée.

Partie I — Les Migrations

1. Qu'est-ce qu'une migration ?

Une migration, c'est un **fichier PHP qui décrit une modification de la structure de la base de données** (création de table, ajout de colonne, suppression d'index, etc.). C'est en quelque sorte le "Git" de ta base de données : chaque migration est un commit qui fait évoluer le schéma.

Au lieu d'écrire du SQL brut (`CREATE TABLE users (...)`), tu écris du PHP avec le Query Builder de Laravel (`Schema::create(...)`), et Laravel traduit ça en SQL adapté à ton moteur de BDD (MySQL, PostgreSQL, SQLite...).

Pourquoi utiliser des migrations plutôt que du SQL direct ou phpMyAdmin ?

Sans migrations	Avec migrations
Chaque dev doit recréer la BDD manuellement	<code>php artisan migrate</code> suffit
Aucun historique des changements	Chaque changement est versionné dans Git
Difficile de revenir en arrière	<code>php artisan migrate:rollback</code>
Risque d'incohérence entre environnements (dev/prod)	Même schéma partout, garanti
Travail d'équipe compliqué	Chacun récupère les migrations de l'autre via Git

2. Anatomie d'un fichier de migration

Reprenons ton fichier comme exemple concret :

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('users');
    }
};
```

Les éléments clés

- `return new class extends Migration` : depuis Laravel 8+, les migrations sont des classes anonymes (plus besoin de leur donner un nom de classe unique, ça évite les conflits).
- `up()` : la méthode qui s'exécute quand tu lances `php artisan migrate`. C'est ici que tu décris ce que tu veux **créer ou modifier**.
- `down()` : la méthode "miroir" qui **annule** ce que fait `up()`. Elle s'exécute avec `php artisan migrate:rollback`.
- `Schema::create('nom_table', function (Blueprint $table) {...})` : crée une nouvelle table. `$table` est l'objet Blueprint qui te permet de définir les colonnes.
- `Schema::table('nom_table', function (Blueprint $table) {...})` : sert à **modifier** une table existante (utilisé dans les migrations ultérieures).

Le nom du fichier compte !

Un fichier de migration s'appelle par exemple :

```
2026_08_19_143022_create_users_table.php
```

Le préfixe `2026_08_19_143022` (année_mois_jour_heureminutessecondes) sert à **ordonner** les migrations. Laravel les exécute dans cet ordre chronologique — essentiel si une table dépend d'une autre (ex: `stages` qui référence `schools` doit être créée après `schools`).

3. Les types de colonnes les plus utilisés

Le `Blueprint` propose une API fluide pour définir tes colonnes :

```
$table->id(); // BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
$table->string('titre', 255); // VARCHAR(255)
$table->text('description'); // TEXT
$table->longText('contenu'); // LONGTEXT
$table->integer('quantite'); // INT
$table->bigInteger('vues'); // BIGINT
$table->unsignedInteger('stock'); // INT UNSIGNED
$table->decimal('prix', 8, 2); // DECIMAL(8,2) — ex: 999999.99
$table->float('note'); // FLOAT
$table->boolean('actif'); // BOOLEAN (TINYINT en MySQL)
$table->date('date_naissance'); // DATE
$table->dateTime('date_evenement'); // DATETIME
$table->timestamp('email_verified_at'); // TIMESTAMP
$table->time('heure_ouverture'); // TIME
$table->json('metadata'); // JSON
$table->uuid('identifiant'); // CHAR(36) pour UUID
$table->enum('statut', ['actif','inactif','suspendu']); // ENUM
$table->foreignId('user_id'); // BIGINT UNSIGNED (pour clé étrangère)
```

Colonnes “raccourcis” très utiles

```
$table->timestamps(); // ajoute created_at + updated_at (nullable, gérés automatiquement par Eloquent)
$table->softDeletes(); // ajoute deleted_at (pour la suppression logique)
$table->rememberToken(); // colonne pour "se souvenir de moi" (VARCHAR(100) nullable)
```

4. Les modificateurs de colonnes

Après avoir défini une colonne, tu peux chaîner des modificateurs :

```
$table->string('email')->unique(); // valeur unique dans la table
$table->string('telephone')->nullable(); // peut être NULL
$table->integer('vues')->default(0); // valeur par défaut
$table->string('slug')->index(); // ajoute un index (recherche plus rapide)
$table->text('bio')->nullable()->comment('Bio libre');
$table->decimal('prix', 10, 2)->unsigned(); // pas de valeurs négatives
$table->string('code')->after('name'); // position de la colonne (MySQL uniquement)
```

⚠ **Important** : l'ordre des modificateurs compte parfois, et tous ne sont pas compatibles avec tous les moteurs (ex: `->after()` ne fonctionne pas sur PostgreSQL).

5. Les clés étrangères (relations entre tables)

C'est LE point crucial pour un projet comme le tien (stagilog avec `dossiers`, `schools`, etc.).

Méthode moderne (recommandée, Laravel 7+)

```
Schema::create('dossiers', function (Blueprint $table) {
    $table->id();
    $table->string('nom_etudiant');
    $table->foreignId('school_id')
        ->constrained()           // suppose une table "schools" avec une clé "id"
        ->onDelete('cascade');    // si l'école est supprimée, ses dossiers aussi
    $table->timestamps();
});
```

`->constrained()` déduit automatiquement : - le nom de la table référencée (`school_id` → table `schools`) - la colonne référencée (`id` par défaut)

Tu peux être explicite si le nom ne correspond pas à la convention :

```
$table->foreignId('school_id')->constrained('schools', 'id');
```

Les actions `onDelete` / `onUpdate`

```
->onDelete('cascade')    // supprime les lignes liées automatiquement
->onDelete('restrict')    // empêche la suppression si des lignes liées existent
->onDelete('set null')    // met la colonne à NULL (nécessite ->nullable())
->onDelete('no action')   // ne fait rien de spécial (comportement par défaut)
```

Méthode “manuelle” (ancienne syntaxe, encore vue dans du vieux code)

```
$table->unsignedBigInteger('school_id');
$table->foreign('school_id')->references('id')->on('schools')->onDelete('cascade');
```

6. Modifier une table existante

Une fois une table créée, tu ne dois **jamais éditer l'ancienne migration** (surtout si elle a déjà été exécutée en production). Tu crées une **nouvelle migration** :

```
php artisan make:migration add_phone_to_users_table --table=users
```

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('phone')->nullable()->after('email');
    });
}

public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn('phone');
    });
}
```

Opérations courantes de modification

```
$table->dropColumn('phone');           // supprimer une colonne
$table->renameColumn('phone', 'telephone'); // renommer
$table->dropForeign(['school_id']);     // supprimer une clé étrangère
$table->dropUnique(['email']);          // supprimer une contrainte unique
$table->dropIndex(['slug']);            // supprimer un index
```

△ Pour modifier le **type** d'une colonne existante (`->change()`), il faut installer le package `doctrine/dbal` :

```
composer require doctrine/dbal
```

```
$table->string('phone', 20)->change();
```

7. Les commandes Artisan indispensables

Commande	Effet
<code>php artisan make:migration create_posts_table</code>	Crée un nouveau fichier de migration vide
<code>php artisan make:migration create_posts_table --create=posts</code>	Crée un fichier pré-rempli avec <code>Schema::create</code>
<code>php artisan make:migration add_status_to_posts_table --table=posts</code>	Pré-rempli pour modifier une table
<code>php artisan migrate</code>	Exécute toutes les migrations en attente
<code>php artisan migrate:status</code>	Affiche l'état de chaque migration (exécutée ou non)
<code>php artisan migrate:rollback</code>	Annule le dernier batch de migrations
<code>php artisan migrate:rollback --step=3</code>	Annule les 3 dernières migrations
<code>php artisan migrate:reset</code>	Annule toutes les migrations
<code>php artisan migrate:refresh</code>	Rollback total + ré-exécution de toutes les migrations
<code>php artisan migrate:fresh</code>	Supprime toutes les tables puis relance les migrations (le plus radical)
<code>php artisan migrate:fresh --seed</code>	Idem + exécute les seeders (données de test)

💡 En développement local (XAMPP comme pour stagilog), `migrate:fresh` est ton meilleur ami quand tu changes souvent de structure — mais **jamais en production** évidemment, ça efface toutes les données !

8. Créer une migration “avec le modèle” (bonne pratique)

Quand tu crées un modèle Eloquent, tu peux générer la migration (et même le contrôleur) en même temps :

```
php artisan make:model Dossier -mcr
```

- `-m` : crée aussi la migration
- `-c` : crée aussi le contrôleur
- `-r` : contrôleur de type “resource” (avec les 7 méthodes CRUD)
- `-f` : ajoute aussi une factory
- `-s` : ajoute aussi un seeder

C’est très efficace pour démarrer rapidement une nouvelle entité.

9. La table `migrations` : comment Laravel sait ce qui a déjà été exécuté

Laravel crée automatiquement une table `migrations` dans ta base de données. Chaque ligne représente un fichier de migration déjà exécuté, avec un numéro de **batch** (groupe d’exécution).

```
+-----+-----+-----+
| id | migration                                | batch |
+-----+-----+-----+
| 1  | 2026_08_19_143022_create_users_table  | 1      |
| 2  | 2026_08_19_143025_create_schools_table| 1      |
| 3  | 2026_08_20_090000_create_dossiers_table| 2      |
+-----+-----+-----+
```

Quand tu lances `php artisan migrate`, Laravel compare les fichiers présents dans `database/migrations` avec cette table, et **n’exécute que les nouveaux fichiers**. C’est pour ça que tu peux relancer `migrate` sans risque : les migrations déjà passées sont ignorées.

10. Exemple concret : construire un mini-schéma pour un projet type “stagilog”

Imaginons trois tables liées : `schools`, `students`, `dossiers`.

```
// 2026_08_19_100000_create_schools_table.php
Schema::create('schools', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->string('password');
    $table->timestamps();
});

// 2026_08_19_100100_create_students_table.php
Schema::create('students', function (Blueprint $table) {
    $table->id();
    $table->foreignId('school_id')->constrained()->onDelete('cascade');
    $table->string('nom');
    $table->string('prenom');
    $table->string('matricule')->unique();
    $table->timestamps();
});

// 2026_08_19_100200_create_dossiers_table.php
Schema::create('dossiers', function (Blueprint $table) {
    $table->id();
    $table->foreignId('student_id')->constrained()->onDelete('cascade');
    $table->string('rapport_path')->nullable();
    $table->enum('statut', ['en_attente', 'valide', 'rejete'])->default('en_attente');
    $table->timestamps();
});
```

Remarque l’**ordre des fichiers** : `schools` avant `students` avant `dossiers`, car chaque table dépend de la précédente via une clé étrangère. Si tu inverses l’ordre, `php artisan migrate` plantera avec une erreur de type “table schools doesn’t exist yet”.

11. Bonnes pratiques à retenir

1. **Une migration = un changement logique.** Ne mélange pas la création de 5 tables sans rapport dans un seul fichier.
2. **Ne modifie jamais une migration déjà exécutée en production.** Crée toujours une nouvelle migration pour ajuster.
3. **Toujours écrire `down()` correctement**, même si tu ne fais jamais de rollback en pratique — ça sert de documentation et de filet de sécurité.
4. **Utilise `foreignId()->constrained()`** plutôt que la syntaxe manuelle : plus lisible et moins d’erreurs.
5. **Pense à l’ordre des dépendances** entre tables (les tables référencées doivent être créées avant).
6. **Utilise des seeders et factories** pour peupler ta base de données de test plutôt que de le faire à la main dans `phpMyAdmin`.
7. **Versions tes migrations avec Git** — c’est tout l’intérêt du système : toute l’équipe reste synchronisée.

12. Aller plus loin sur les migrations

- Les **seeders** (`php artisan make:seeder`) : pour insérer des données de test après une migration.
- Les **factories** : pour générer des données fictives réalistes (utile pour tester ton appli stagilog avec des dizaines de faux étudiants).
- Les migrations sur **PostgreSQL/SQLite** : la même syntaxe fonctionne, Laravel adapte le SQL généré.
- La commande `php artisan schema:dump` : pour “compresser” un long historique de migrations en un seul fichier SQL de référence (utile sur de gros projets anciens).

Partie II — La connexion à la base de données

13. Le fichier `.env`

Laravel ne stocke **jamais** les identifiants de connexion directement dans le code. Tout passe par le fichier `.env`, à la racine du projet (jamais versionné dans Git — il doit figurer dans `.gitignore`) :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=stagilog
DB_USERNAME=root
DB_PASSWORD=
```

Sur un environnement XAMPP local (comme celui utilisé pour stagilog), c'est typiquement `DB_HOST=127.0.0.1`, `DB_PORT=3306`, et `DB_USERNAME=root` avec un mot de passe vide par défaut.

△ Après toute modification du `.env`, si tu utilises le cache de configuration, il faut vider le cache :

```
php artisan config:clear
```

14. Le fichier `config/database.php`

Ce fichier lit les valeurs du `.env` et définit **toutes les connexions disponibles** pour l'application :

```
'default' => env('DB_CONNECTION', 'mysql'),

'connections' => [
    'mysql' => [
        'driver' => 'mysql',
        'host' => env('DB_HOST', '127.0.0.1'),
        'port' => env('DB_PORT', '3306'),
        'database' => env('DB_DATABASE', 'forge'),
        'username' => env('DB_USERNAME', 'forge'),
        'password' => env('DB_PASSWORD', ''),
        'charset' => 'utf8mb4',
        'collation' => 'utf8mb4_unicode_ci',
        'prefix' => '',
        'strict' => true,
        'engine' => null,
    ],

    'sqlite' => [
        'driver' => 'sqlite',
        'database' => database_path('database.sqlite'),
        'prefix' => '',
    ],

    // pgsql, sqlsrv... même logique
],
```

Pourquoi cette séparation `.env` / `config/database.php` ?

- Le `.env` change **selon l'environnement** (dev, staging, production) sans jamais toucher au code.
- `config/database.php` reste identique partout — il lit juste les variables via `env()`.
- Chaque développeur de l'équipe a son propre `.env` local (ex: mot de passe MySQL différent), sans conflit Git.

Utiliser plusieurs connexions (ex: une BDD secondaire de logs)

```
// config/database.php
'connections' => [
    'mysql' => [ /* ... */ ],
    'logs' => [
        'driver' => 'mysql',
        'host' => env('DB_LOGS_HOST', '127.0.0.1'),
        'database' => env('DB_LOGS_DATABASE', 'logs_db'),
        // ...
    ],
],
```

```
// Utilisation dans le code
DB::connection('logs')->table('activities')->insert([...]);

// Ou dans un modèle Eloquent
class LogEntry extends Model
{
    protected $connection = 'logs';
}
```

15. Tester et utiliser la connexion

```
php artisan migrate      # vérifie implicitement que la connexion fonctionne
php artisan db           # ouvre un client CLI connecté à ta base (MySQL/psql/sqlite)
php artisan tinker       # console interactive PHP, pratique pour tester des requêtes
```

Dans Tinker :

```
DB::connection()->getPdo(); // vérifie que la connexion est établie sans erreur
DB::select('select 1+1 as somme');
```

Requêtes brutes avec la façade `DB`

Même sans passer par Eloquent, tu peux interroger directement :

```
use Illuminate\Support\Facades\DB;

$dossiers = DB::table('dossiers')->where('statut', 'valide')->get();

DB::insert('insert into schools (name, email) values (?, ?)', ['Lycée Central', 'contact@lycee.bj']);

DB::table('users')->where('id', 1)->update(['name' => 'Golo']);

DB::table('users')->where('id', 1)->delete();
```

Partie III — Le routage (Routes)

16. Où définir les routes ?

- `routes/web.php` : routes destinées au navigateur (avec sessions, cookies, protection CSRF).
- `routes/api.php` : routes destinées à une API (pas de sessions, généralement authentifiées par token, préfixées automatiquement par `/api`).
- `routes/console.php` : commandes Artisan personnalisées.

17. Routes de base

```
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dossiers', [DossierController::class, 'index']);
Route::post('/dossiers', [DossierController::class, 'store']);
Route::put('/dossiers/{id}', [DossierController::class, 'update']);
Route::patch('/dossiers/{id}', [DossierController::class, 'update']);
Route::delete('/dossiers/{id}', [DossierController::class, 'destroy']);
```

Chaque verbe HTTP correspond à une action : `GET` (lire), `POST` (créer), `PUT / PATCH` (modifier), `DELETE` (supprimer).

18. Paramètres de route

```
// Paramètre obligatoire
Route::get('/dossiers/{id}', [DossierController::class, 'show']);

// Paramètre optionnel
Route::get('/dossiers/{id?}', function ($id = null) {
    return $id ?? 'Aucun ID fourni';
});

// Contrainte sur le paramètre (regex)
Route::get('/dossiers/{id}', [DossierController::class, 'show'])
    ->where('id', '[0-9]+');

// Plusieurs paramètres
Route::get('/schools/{school}/students/{student}', [StudentController::class, 'show']);
```

19. Routes nommées

Nommer une route permet d'y faire référence sans coder l'URL en dur — très utile si l'URL change un jour :

```
Route::get('/dossiers/{id}', [DossierController::class, 'show'])->name('dossiers.show');
```

```
// Dans une vue Blade ou un contrôleur
$url = route('dossiers.show', ['id' => 5]); // génère /dossiers/5
return redirect()->route('dossiers.show', ['id' => 5]);
```

20. Routes ressources (Resource Routes)

Pour un CRUD complet, plutôt que d'écrire 7 routes à la main, une seule ligne suffit :

```
Route::resource('dossiers', DossierController::class);
```

Cela génère automatiquement :

Verbe HTTP	URI	Action	Nom de route
GET	/dossiers	index	dossiers.index
GET	/dossiers/create	create	dossiers.create
POST	/dossiers	store	dossiers.store
GET	/dossiers/{id}	show	dossiers.show
GET	/dossiers/{id}/edit	edit	dossiers.edit
PUT/PATCH	/dossiers/{id}	update	dossiers.update
DELETE	/dossiers/{id}	destroy	dossiers.destroy

Pour une API (sans `create` / `edit`, qui sont des vues HTML) :

```
Route::apiResource('dossiers', DossierController::class);
```

21. Groupes de routes

Pour appliquer un préfixe, un middleware ou un namespace à plusieurs routes d'un coup :

```
Route::prefix('admin')->middleware('auth')->group(function () {
    Route::get('/dashboard', [AdminController::class, 'dashboard']);
    Route::resource('users', AdminUserController::class);
});
// -> génère /admin/dashboard, /admin/users, etc., protégées par le middleware "auth"
```

```
// Groupe avec un préfixe de nom de route
Route::name('admin.')->group(function () {
    Route::get('/dashboard', [AdminController::class, 'dashboard'])->name('dashboard');
    // nom final : admin.dashboard
});
```

22. Middlewares sur les routes

Un middleware filtre ou modifie une requête avant qu'elle n'atteigne le contrôleur (ex: vérifier qu'un utilisateur est connecté) :

```
Route::get('/dashboard', [DashboardController::class, 'index'])->middleware('auth');

Route::middleware(['auth', 'verified'])->group(function () {
    // routes accessibles seulement aux utilisateurs connectés et vérifiés
});
```

Pour ton projet stagilog, un middleware personnalisé `auth:school` (guard spécifique) pourrait protéger l'espace de connexion des écoles.

23. Voir toutes les routes de l'application

```
php artisan route:list
php artisan route:list --name=dossiers # filtrer par nom
```

Partie IV — Les Modèles Eloquent

24. Qu'est-ce qu'Eloquent ?

Eloquent est l'**ORM** (Object-Relational Mapper) de Laravel : chaque modèle PHP représente une table de la base de données, et chaque instance du modèle représente une ligne de cette table. Il permet de manipuler la base de données avec des objets PHP plutôt qu'avec du SQL.

25. Créer un modèle

```
php artisan make:model Dossier
php artisan make:model Dossier -m # + la migration associée
php artisan make:model Dossier -mcr # + migration, contrôleur resource
```

Par convention, un modèle `Dossier` (singulier) correspond à une table `dossiers` (pluriel, snake_case). Tu peux forcer un autre nom si besoin :

```
class Dossier extends Model
{
    protected $table = 'mes_dossiers'; // si le nom ne suit pas la convention
}
```

26. Structure de base d'un modèle

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Dossier extends Model
{
    // Colonnes autorisées pour l'assignation en masse (create/update)
    protected $fillable = ['student_id', 'rapport_path', 'statut'];

    // OU la liste inverse : colonnes interdites
    // protected $guarded = ['id'];

    // Conversion automatique de types
    protected $casts = [
        'statut' => 'string',
        'date_soumission' => 'datetime',
        'metadata' => 'array', // convertit une colonne JSON en tableau PHP
    ];

    // Désactive created_at/updated_at si la table ne les a pas
    // public $timestamps = false;
}
```

⚠ **fillable** vs **guarded** : pourquoi c'est important pour la sécurité

Sans l'un des deux, Eloquent lève une exception `MassAssignmentException` si tu fais :

```
Dossier::create($request->all());
```

fillable protège contre l'**assignation de masse malveillante** — par exemple, empêcher qu'un utilisateur envoie `is_admin=1` dans un formulaire pour s'auto-promouvoir administrateur.

27. Opérations CRUD avec Eloquent

```
// CREATE
$dossier = Dossier::create([
    'student_id' => 3,
    'rapport_path' => 'rapports/dossier3.pdf',
    'statut' => 'en_attente',
]);

// READ
$tous = Dossier::all();
$un = Dossier::find(5); // par clé primaire
$un = Dossier::findOrFail(5); // lève une 404 si absent
$filtres = Dossier::where('statut', 'valide')->get();
$premier = Dossier::where('statut', 'valide')->first();

// UPDATE
$dossier = Dossier::find(5);
$dossier->statut = 'valide';
$dossier->save();

// ou en une ligne :
Dossier::where('id', 5)->update(['statut' => 'valide']);

// DELETE
$dossier = Dossier::find(5);
$dossier->delete();

// ou :
Dossier::destroy(5); // par id
Dossier::destroy([5, 6, 7]); // plusieurs ids
```

28. Le Query Builder d'Eloquent (requêtes avancées)

```
Dossier::where('statut', 'valide')
->where('created_at', '>=', now()->subDays(7))
->orderBy('created_at', 'desc')
->limit(10)
->get();

Dossier::whereIn('statut', ['valide', 'en_attente'])->get();
Dossier::whereNull('rapport_path')->get();
Dossier::count();
Dossier::paginate(15); // pagination automatique avec liens
```

29. Les relations entre modèles

C'est le cœur de la puissance d'Eloquent : représenter les clés étrangères comme des relations navigables en PHP.

hasOne / **belongsTo** (relation 1-1 ou 1-n côté "enfant")

```

class School extends Model
{
    public function students()
    {
        return $this->hasMany(Student::class); // une école a plusieurs étudiants
    }
}

class Student extends Model
{
    public function school()
    {
        return $this->belongsTo(School::class); // un étudiant appartient à une école
    }
}

```

Utilisation :

```

$school = School::find(1);
$school->students; // collection de tous les étudiants de cette école

$student = Student::find(1);
$student->school->name; // accède directement au nom de l'école liée

```

hasMany (1 vers plusieurs)

```

class Student extends Model
{
    public function dossiers()
    {
        return $this->hasMany(Dossier::class);
    }
}

$student->dossiers; // tous les dossiers de cet étudiant
$student->dossiers()->where('statut', 'valide')->get(); // requête affinée

```

belongsToMany (relation many-to-many, avec table pivot)

```

class Student extends Model
{
    public function courses()
    {
        return $this->belongsToMany(Course::class); // suppose une table pivot "course_student"
    }
}

$student->courses; // tous les cours de l'étudiant
$student->courses()->attach($courseId); // ajouter une relation
$student->courses()->detach($courseId); // retirer une relation
$student->courses()->sync([1, 2, 3]); // remplace toutes les relations existantes

```

Chargement “eager loading” (éviter le problème N+1)

```

// ✖ Mauvais : 1 requête pour les étudiants + N requêtes (une par étudiant pour l'école)
$students = Student::all();
foreach ($students as $s) { echo $s->school->name; }

// ✔ Bon : 2 requêtes seulement, quel que soit le nombre d'étudiants
$students = Student::with('school')->get();
foreach ($students as $s) { echo $s->school->name; }

```

30. Accesseurs et mutateurs (personnaliser la lecture/écriture)

```

class Student extends Model
{
    // Accesseur : transforme la donnée à la LECTURE
    protected function nomComplet(): Attribute
    {
        return Attribute::make(
            get: fn () => "{$this->prenom} {$this->nom}",
        );
    }

    // Mutateur : transforme la donnée à l'ÉCRITURE
    protected function email(): Attribute
    {
        return Attribute::make(
            set: fn (string $value) => strtolower($value),
        );
    }
}

$student->nom_complet;           // utilise l'accesseur automatiquement
$student->email = 'TEST@MAIL.COM'; // stocké en minuscules grâce au mutateur

```

31. Scopes locaux (requêtes réutilisables)

```

class Dossier extends Model
{
    public function scopeValides($query)
    {
        return $query->where('statut', 'valide');
    }
}

// Utilisation, très lisible :
Dossier::valides()->get();
Dossier::valides()->where('student_id', 3)->get();

```

Partie V — Comment tout s'articule (le flux MVC)

32. De la requête HTTP à la réponse

1. Une requête arrive (ex: `GET /dossiers/5`).
2. Laravel consulte `routes/web.php`, trouve la route correspondante, et applique les **middlewares** définis (auth, CSRF, etc.).
3. Le **contrôleur** associé est appelé (ex: `DossierController@show`).
4. Le contrôleur utilise le **modèle Eloquent** pour interroger la base (`Dossier::findOrFail(5)`), qui passe elle-même par la **connexion** configurée dans `config/database.php`.
5. Le contrôleur retourne une **vue** (Blade) ou une réponse **JSON** (pour une API).

```

// routes/web.php
Route::get('/dossiers/{id}', [DossierController::class, 'show'])->name('dossiers.show');

// app/Http/Controllers/DossierController.php
class DossierController extends Controller
{
    public function show($id)
    {
        $dossier = Dossier::with('student')->findOrFail($id);
        return view('dossiers.show', compact('dossier'));
    }
}

```

C'est exactement le schéma que tu pourrais appliquer si tu portes ton projet **stagilog** (actuellement en PHP/MySQL natif) vers Laravel : chaque table (`schools`, `students`, `dossiers`) devient une migration + un modèle, chaque page devient une route + un contrôleur.

Conclusion et ressources

-
- Les **seeders** (`php artisan make:seeder`) et **factories** pour peupler ta base de test.
 - La commande `php artisan tinker` pour expérimenter en direct avec tes modèles et requêtes.
 - La documentation officielle : laravel.com/docs reste la référence la plus à jour, en particulier pour les fonctionnalités ajoutées après ce cours.

Si tu veux, je peux te préparer un jeu de migrations + modèles + routes complet et prêt à l'emploi pour ton projet **stagilog** (schools, students, dossiers, avec toutes les relations), adapté à ce que tu as déjà commencé en PHP/MySQL pur — soit en migrations Laravel si tu envisages de migrer stagilog vers ce framework, soit en SQL brut si tu restes sur PHP natif.